

Zeal College of Engineering and Research

Subject: LP-V

Name: Jenil Girish Rathod

Class: BE

Div: B

Batch: B1

Roll No: B22009

Group A: Practical No:2

Program Statement: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Implemented Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <omp.h>
#include <iomanip>

using namespace std;

// Thresholds and tuning
const int BUBBLE_SORT_SKIP_THRESHOLD = 2000; // skip bubble for large n
const int MERGE_SORT_TASK_THRESHOLD = 1000; // only create tasks for segments
larger than this
// ===== UTILITY =====
void generateArray(vector<int> &arr) {
    for (size_t i = 0; i < arr.size(); ++i)
        arr[i] = rand() % 100000; // wider range
}
// ===== SEQUENTIAL BUBBLE SORT =====
// Standard optimized bubble sort with early exit
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
    }
    if (!swapped) // already sorted
```

```

        break;
    }
}
// ===== PARALLEL BUBBLE SORT (Odd-Even)
// Uses parallel for in even/odd phases. Uses atomic increments to detect
// whether any swap happened across threads so we can exit early.
void parallelBubbleSort(int arr[], int n) {
    for (int i = 0; i < n; ++i) {
        int swapped = 0;

        // Even phase
        #pragma omp parallel for
        for (int j = 0; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                #pragma omp atomic
                ++swapped;
            }
        }

        // Odd phase
        #pragma omp parallel for
        for (int j = 1; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                #pragma omp atomic
                ++swapped;
            }
        }

        if (!swapped) // no swaps in both phases -> sorted
            break;
    }
}
// ===== MERGE FUNCTION (dynamic buffers) =====
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;
    // allocate temporary buffers dynamically
    int *L = new int[n1];
    int *R = new int[n2];

    for (int i = 0; i < n1; ++i)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; ++j)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])

```

```

        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}
while (i < n1)
    arr[k++] = L[i++];
while (j < n2)
    arr[k++] = R[j++];
delete[] L;
delete[] R;
}
// ===== SEQUENTIAL MERGE SORT =====
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
// ===== PARALLEL MERGE SORT =====
// Uses OpenMP tasks but only for sufficiently large partitions to avoid
// creating too many tasks which hurts performance.
void parallelMergeSort(int arr[], int l, int r) {
    if (l >= r) return;

    int size = r - l + 1;
    if (size <= MERGE_SORT_TASK_THRESHOLD) {
        // For small partitions, fall back to sequential merge sort
        mergeSort(arr, l, r);
        return;
    }

    int m = l + (r - l) / 2;

    #pragma omp task shared(arr) firstprivate(l,m)
    {
        parallelMergeSort(arr, l, m);
    }

    #pragma omp task shared(arr) firstprivate(m,r)
    {
        parallelMergeSort(arr, m + 1, r);
    }

    #pragma omp taskwait
    merge(arr, l, m, r);
}
// ===== MAIN =====

```

```

int main() {
    srand((unsigned)time(nullptr));

    int n;
    cout << "Enter number of elements: ";
    if (!(cin >> n) || n <= 0) {
        cout << "Invalid input. Exiting.\n";
        return 1;
    }

    // Use vectors sized to n so memory is minimal and dynamic
    vector<int> arr(n), arr1(n), arr2(n), arr3(n), arr4(n);
    generateArray(arr);

    // Create copies for each algorithm
    arr1 = arr;
    arr2 = arr;
    arr3 = arr;
    arr4 = arr;

    double start, end;
    cout << fixed << setprecision(6);

    // Sequential Bubble Sort (skipped for large n)
    if (n <= BUBBLE_SORT_SKIP_THRESHOLD) {
        start = omp_get_wtime();
        bubbleSort(arr1.data(), n);
        end = omp_get_wtime();
        cout << "Sequential Bubble Sort Time: " << (end - start) << " seconds\n";
    } else {
        cout << "Sequential Bubble Sort Time: Skipped for n > " <<
BUBBLE_SORT_SKIP_THRESHOLD << "\n";
    }

    // Parallel Bubble Sort (skipped for large n)
    if (n <= BUBBLE_SORT_SKIP_THRESHOLD) {
        start = omp_get_wtime();
        parallelBubbleSort(arr2.data(), n);
        end = omp_get_wtime();
        cout << "Parallel Bubble Sort Time: " << (end - start) << " seconds\n";
    } else {
        cout << "Parallel Bubble Sort Time: Skipped for n > " <<
BUBBLE_SORT_SKIP_THRESHOLD << "\n";
    }

    // Debug message to verify merge sort section runs
    cout << "\nDebug: Starting Merge Sorts...\n";

    // Sequential Merge Sort
    start = omp_get_wtime();

```

```

mergeSort(arr3.data(), 0, n - 1);
end = omp_get_wtime();
cout << "Sequential Merge Sort Time: " << (end - start) << " seconds\n";

// Parallel Merge Sort
start = omp_get_wtime();
#pragma omp parallel
{
    #pragma omp single
    {
        parallelMergeSort(arr4.data(), 0, n - 1);
    }
}
end = omp_get_wtime();
cout << "Parallel Merge Sort Time: " << (end - start) << " seconds\n";

return 0;
}

```

How to compile: g++ -fopenmp Practical2.cpp -O2 -std=c++11 -o program

OR

How to compile: g++ -fopenmp Practical2.cpp -o program

How to run: program.exe

OUTPUT:

```

C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>g++ -fopenmp Practical2.cpp -o Practical2

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>Practical2.exe
Enter number of elements: 1000
Sequential Bubble Sort Time: 0.002000 seconds
Parallel Bubble Sort Time: 0.124000 seconds

Debug: Starting Merge Sorts...
Sequential Merge Sort Time: 0.001000 seconds
Parallel Merge Sort Time: 0.000000 seconds

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>

```